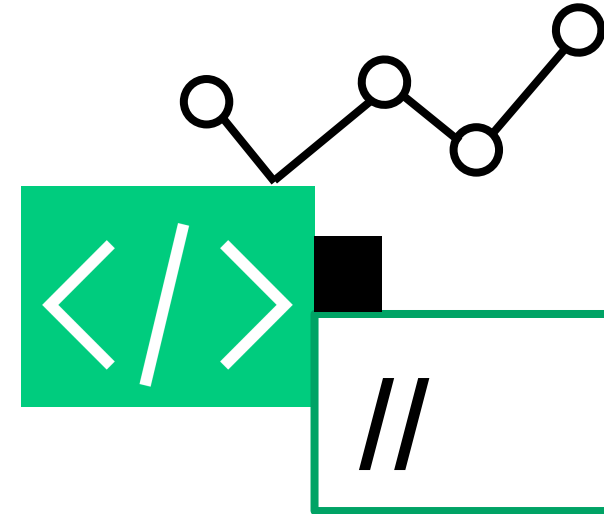


Introducción a TensorFlow y PyTorch

03

Introducción a PyTorch



Definir un modelo

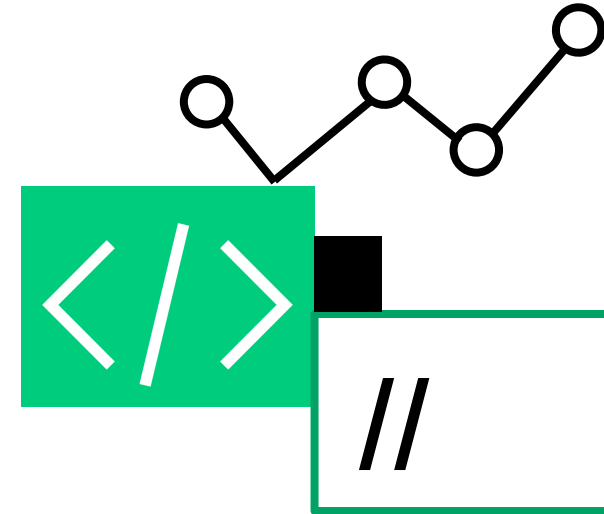
Para crear un modelo de red neuronal, hay que definirlo a través de una clase que va a heredar de ***torch.nn.Module***. Es la clase base en PyTorch para todos los modelos de redes neuronales.

Proporciona una estructura estándar para construir, entrenar y evaluar redes neuronales.

```
import torch.nn as nn

class MyNeuralNetwork(nn.Module):
    def __init__(self):
        super(MyNeuralNetwork, self).__init__()
        #Neural Network Building

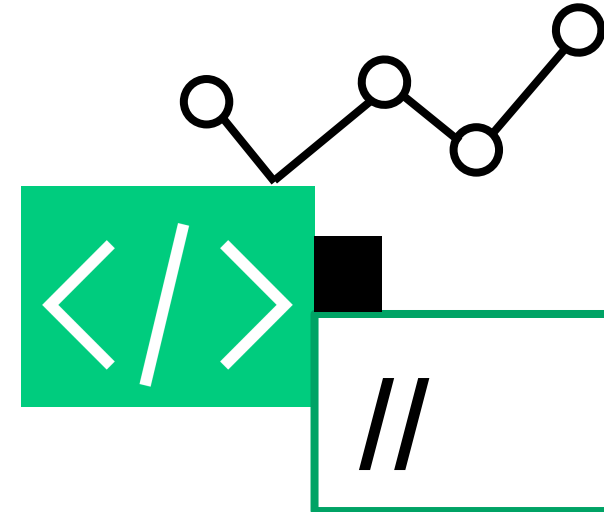
    def forward(self, x):
        #Forward operations
        return x
```



Definir un modelo

Código para crear una pequeña red neuronal (MLP).

```
import torch.nn as nn
class MyNeuralNetwork(nn.Module):
    def __init__(self):
        super(MyNeuralNetwork, self).__init__()
        self.layer1 = nn.Linear(8, 10)
        self.layer2 = nn.Linear(10, 3)
    def forward(self, x):
        x = self.layer1(x)
        x = self.layer2(x)
        return x
```

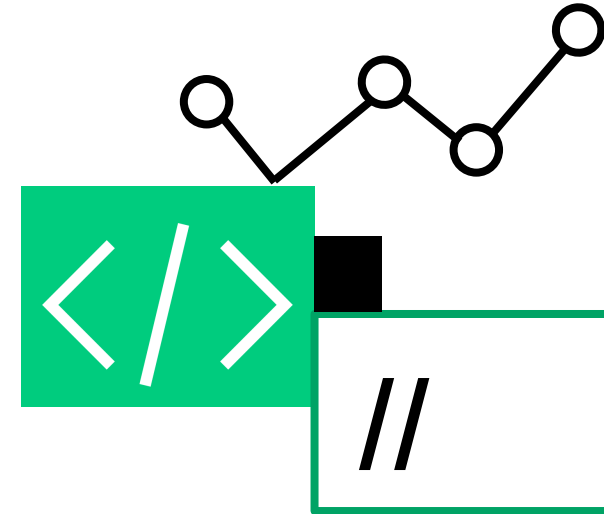


Definir un modelo

Código para crear una pequeña red neuronal (MLP).

```
import torch.nn as nn
class MyNeuralNetwork(nn.Module):
    def __init__(self):
        super(MyNeuralNetwork, self).__init__()
        self.layer1 = nn.Linear(8, 10)
        self.layer2 = nn.Linear(10, 3)
    def forward(self, x):
        x = self.layer1(x)
        x = self.layer2(x)
        return x
```

```
def forward(self, x):
    x = self.layer1(x)
    x = torch.relu()
    x = self.layer2(x)
    x = torch.softmax()
    return x
```

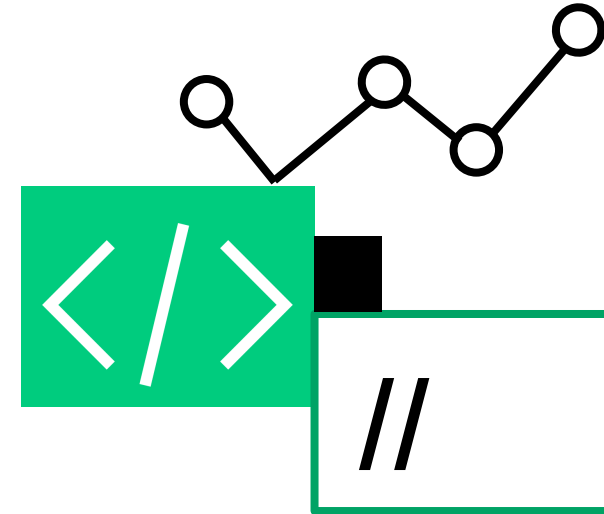


Función de pérdida

Seleccionar la función de **pérdida o error (loss)**, en Pytorch se encuentran definidas en ***torch.nn***.

- **torch.nn.BCELoss** (binary cross entropy) para clasificación binaria.
- **torch.nn.CrossEntropyLoss** (cross entropy) para clasificación multiclase.
- **torch.nn.MSELoss** (mean squared error) para problemas de regresión.

```
criterion = nn.CrossEntropyLoss()
```



Algoritmo de optimización

Seleccionar y parametrizar el algoritmo de optimización. En Pytorch los optimizadores se encuentran definidos en ***torch.optim***.

- **torch.optim.SGD()** implementa el Descenso de Gradiente Estocástico.
- **torch.optim.Adam()** implementa Adam.

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

- SGD incorpora el parámetro **momentum**. Esto ayuda a acelerar el aprendizaje y a evitar quedarse atrapado en mínimos locales al tener en cuenta el gradiente de los pasos anteriores.

¡Muchas gracias!